

The Quantum Threat - Shor's algorithm

MFC Seminar WS 2023/2024

Sebastian David Albu

Institute of Applied Information Processing and Communications (IAIK),
Graz University of Technology, Graz, Austria
`sebastian.albu@student.tugraz.at`

Abstract. Public-key cryptography relies on the complexity of hard mathematical problems, such as the IFP (*Integer Factorization Problem*) and the DLP (*Discrete Logarithm Problem*). Using quantum computers to attack those concepts comes with an exponential speedup over classical computers. The aim of this seminar project is to delve into quantum computing fundamentals and Shor's algorithm for Factorization and DLP.

Keywords: Quantum Computing · Shor's Algorithm · Quantum Fourier Transform (QFT) · Discrete Logarithm Problem (DLP) · Integer Factorization Problem (IFP) · Number Field Sieve (NFS) and General Number Field Sieve (GNFS) · Quantum Threat.

Table of Contents

1	Preliminaries	3
1.1	Mathematical Preliminaries	3
1.2	Quantum Mechanics Preliminaries	4
1.3	Quantum Gates - CNOT and Hadamard Gates	9
2	Shor's Algorithms	12
2.1	Factorization	12
2.2	Recovering discrete logarithms for the Discrete Logarithm Problem (DLP)	14
2.3	Recovering discrete logarithms with the Number Field Sieve (NFS) approach	15
2.4	Comparison to Shor's Algorithm and Analysis	17
3	Discussion	18

1 Preliminaries

Mathematics is undoubtedly the backbone of cryptography. The security aspects of cryptography rely almost exclusively on our beliefs in number and complexity theory. Nonetheless, when we talk about post-quantum cryptography, we also need to cover the basics of how a quantum computer works. In this chapter, we will first cover the needed mathematic notations, and afterward we will cover the basics of a quantum computer.

1.1 Mathematical Preliminaries

As already mentioned, we heavily rely on some fundamental aspects of number theory in cryptography. In this paper, we will focus on two concrete problems which are believed to be *hard* to solve. A hard problem is infeasible to solve in polynomial time on a regular computer.

Definition 1 (Group). *A group is a set G together with a binary operation $*$: $G \times G \rightarrow G$ such that the following is satisfied:*

$$\forall a, b, c \in G : (a * b) * c = a * (b * c), \quad (\text{associative}) \quad (1)$$

$$\exists e \in G \forall a \in G : e * a = a * e = a, \quad (\text{identity element}) \quad (2)$$

$$\forall a \in G \exists a' \in G : a * a' = a' * a = e. \quad (\text{inverses}) \quad (3)$$

An element $g \in G$ is called a *generator* of G if we can perform the binary operation until we have reached every element inside the *finite cyclic group*. A group is called *finite* if it has a finite number of elements. A group is called *cyclic*, if it is created by a generator g , on which the group operation is applied until it cycles back to g . Such a group is then denoted as $G = \langle g \rangle$.

The *group order* of G is the number of elements inside the group. The *element order* is calculated by taking one element inside G and applying the group operation until it reaches the *neutral element*, which is the identity element as seen in equation (2).

A *semiprime number* is an integer which is the product of two prime numbers p and q .

$\mathbb{Z}/n\mathbb{Z}$ represents the set of integers modulo n , denoted as the integers from 0 to $n - 1$ in modular arithmetic. It is often referred to as the integers modulo n or the *finite field* of order n if n is prime.

Definition 2 (Integer Factorization Problem). *Given a non-prime number $n \in \mathbb{N}$, the Integer Factorization Problem (IFP) consists of finding $p, q \in \mathbb{P}$ such that $n = p \cdot q$.*

The IFP is believed to be difficult to solve for very large primes p and q . In that case, the IFP becomes a so-called *hard problem*. Due to this believed complexity we get one of the most well-known public key encryption schemes,

named after its creators, the *RSA* encryption scheme [7]. A noteworthy mention is that often the pure form of RSA is mistakenly used and then the security aspects of it dwindle a bit. These errors are known to be called the errors of using “Textbook RSA” [1].

Definition 3 (Discrete Logarithm Problem). *Given a finite cyclic group $(G, *)$ with generator g and let $x \in G$. The Discrete Logarithm Problem (DLP) consists of finding $r \in \mathbb{N}$ such that*

$$x = \underbrace{g * \dots * g}_{r \text{ times}} =: g^r.$$

For well-chosen groups, the DLP is believed to be hard. A typical example of a good group is $(\mathbb{Z}_p^\times, \cdot)$ for a large prime p . On the other hand, solving the DLP in $(\mathbb{Z}_p, +)$ is easy, as it is equivalent to finding modulo inverses:

$$x = \underbrace{g + \dots + g}_{r \text{ times}} = r \cdot g \implies r = x \cdot g^{-1} \pmod{p}.$$

One prominent use-case of the DLP is the *Diffie-(Merkle)-Hellman* key exchange protocol [5].

1.2 Quantum Mechanics Preliminaries

Having covered some mathematical aspects, there are still some fundamentals to quantum computing, which need to be addressed.

Quantum mechanical notation for notions from Linear Algebra The n -dimensional Hilbert space $\mathcal{H}$ is a complex vector space with an inner product

$$\langle \cdot, \cdot \rangle : \mathcal{H}^* \times \mathcal{H} \rightarrow \mathbb{C}, \quad (4)$$

where $\mathcal{H}^*$ denotes the *dual* or *conjugate space* to $\mathcal{H}$. In the context of quantum mechanics, typically the Dirac notation (also called bra-ket notation) is used in order to avoid cluttered matrix notation.

In the Dirac notation, a vector in $\mathcal{H}$ is denoted by $|\psi\rangle$ (*ket*), and its dual vector is by denoted $\langle\psi|$ (*bra*):

$$|\psi\rangle = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_n \end{bmatrix} \in \mathcal{H} \quad \text{and} \quad \langle\psi| = [\alpha_1^* \ \alpha_2^* \ \dots \ \alpha_n^*] \in \mathcal{H}^*, \quad (5)$$

where α_i^* is the complex conjugate of $\alpha_i \in \mathbb{C}$. The inner product between two vectors $\langle\phi|$ and $|\psi\rangle$ is given by

$$\langle\phi|\psi\rangle := [\beta_1^* \ \beta_2^* \ \dots \ \beta_n^*] \cdot \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_n \end{bmatrix} = \alpha_1 \beta_1^* + \dots + \alpha_n \beta_n^*. \quad (6)$$

The tensor product of the two vectors $|\phi\rangle$ and $|\psi\rangle$ is given by

$$|\phi\rangle \otimes |\psi\rangle := \begin{bmatrix} \beta_1 \\ \vdots \\ \beta_n \end{bmatrix} \otimes \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_n \end{bmatrix} = \begin{bmatrix} \beta_1 \cdot |\psi\rangle \\ \vdots \\ \beta_n \cdot |\psi\rangle \end{bmatrix}. \quad (7)$$

Note that sometimes $|\phi\rangle|\psi\rangle$ is used as an abbreviated notation for the tensor product $|\phi\rangle \otimes |\psi\rangle$. Finally, the inner product between $|\phi\rangle$ and $A|\psi\rangle$, or equivalently between $A^\dagger|\phi\rangle$ and $|\psi\rangle$, is denoted by

$$\langle\phi|A|\psi\rangle,$$

where $A \in \mathbb{C}^{n \times n}$ and $A^\dagger := (A^*)^T$ denotes the adjoint or Hermitian conjugate of A .

The four postulates of Quantum Mechanics Usually, classical schemes of cryptography do not (or rather rarely) cross paths with quantum mechanics. Therefore it is needed to first establish the "ground rules". The following four postulates of quantum mechanics are taken from Ahlström and Dizdarevic [3], and we will in the following refer to that publication unless stated otherwise.

1. **Quantum states:** Every quantum system is associated with a Hilbert space $\mathcal{H}$, called the state space. The system is completely described by its state vectors. Those state vectors shall be normalized, that is, their length is one:

$$|||\psi\rangle|| = \sqrt{\langle\psi|\psi\rangle} = 1.$$

This is particularly useful for the probability of the state measuring.

2. **Evolution of a closed system:** The evolution of a *closed* system, that is, a system that does not interact with the rest of the universe, is described by unitary transformations. This unitary transformation is given by a matrix U such that

$$UU^\dagger = U^\dagger U = \mathbf{I}, \quad (8)$$

where $\mathbf{I}$ denotes the identity matrix. In particular, two states $|\psi\rangle$ and $|\phi\rangle$ at two different times t_1 and t_2 are related by

$$|\phi\rangle = U|\psi\rangle,$$

where $|\phi\rangle$ is again properly normalized:

$$|||\phi\rangle|| = ||U|\psi\rangle|| = \sqrt{\langle\psi|U^\dagger U|\psi\rangle} = \sqrt{\langle\psi|\mathbf{I}|\psi\rangle} = \sqrt{\langle\psi|\psi\rangle} = 1. \quad (9)$$

These transformations ensure that the probabilities associated with different quantum states remain unchanged. This means that as the system evolves over time according to these transformations, its behavior is both reversible and deterministic. In other words, we can predict how the system will change

over time with certainty, and we can also reverse this process to understand how it reached a particular state.

Note that the unitary evolution reflects the conservation of information only in a closed quantum system. A closed system is not really useful to science, as the scientist will want to look at the quantum system to see what happens. However, when the laboratory equipment interacts with the closed system, it no longer evolves unitary.

3. **Quantum measurements:** Performing a quantum measurement is done by applying a series of measurement operators $\{M_k\}_{k=0}^n$, where n is the number of outcomes that can come from measuring the quantum system. $|\psi\rangle$ here represents the state of the quantum system just before the measurement. This state vector is a vector in a Hilbert space $\mathcal{H}$ that describes the possible states of the quantum system.

A system being in the state $|\psi\rangle$ right before the measurement will have a certain probability of yielding the i :th measurement, determined by

$$P(k = i) = \langle\psi|M_i^\dagger M_i|\psi\rangle, \quad (10)$$

where $\langle\psi|M_i^\dagger M_i|\psi\rangle$ denotes the inner product. This expression involves the inner product of the state $|\psi\rangle$ with the adjoint (Hermitian conjugate) of the measurement operator $M_i^\dagger M_i$ multiplied by the measurement operator M_i . This probability expression involves the concept of *quantum superposition*, which will be discussed in Section 1.2. Before measurement, the system can exist in a linear combination of different states, and the measurement process "collapses" the state to one of the possible outcomes.

After the measurement the system has been put into the new state

$$\frac{M_i|\psi\rangle}{\sqrt{\langle\psi|M_i^\dagger M_i|\psi\rangle}} \quad (11)$$

This ensures that the post-measurement state is normalized, maintaining the probabilities. This observation of maintaining the probabilities is essential for measurements in a quantum system!

In summary, quantum measurements involve probabilistic outcomes, and the probabilities are determined by the quantum state and the set of measurement operators. The post-measurement state reflects the system's state after the measurement process.

4. **Composite states** If multiple quantum systems are combined, the composite state space $\mathcal{H}$ is given by the tensor product of each individual system's state space. If the composite space is built up by n spaces, and if the k :th space is the state $|\psi_k\rangle$. The state for a composite state space is

$$|\psi_1\rangle \otimes |\psi_2\rangle \otimes \dots \otimes |\psi_n\rangle = |\psi_1, \psi_2, \dots, \psi_n\rangle = |\psi_1 \psi_2 \dots \psi_n\rangle, \quad (12)$$

whereas $\otimes$ is the tensor product.

The tensor product of quantum states is a way to combine the states of

multiple quantum systems to form a composite state. If we combine multiple quantum systems, the composite state space is given by the tensor product of each individual system's state space. If our composite space is built up by n spaces, and if the k :th space is the state $|\psi_k\rangle$, then the state for a composite state space is

$$|\psi_1\rangle \otimes |\psi_2\rangle \otimes \dots \otimes |\psi_n\rangle. \quad (13)$$

This can also be written more compactly as

$$|\psi_1, \psi_2, \dots, \psi_n\rangle = |\psi_1 \psi_2 \dots \psi_n\rangle. \quad (14)$$

The overall tensor product is obtained by taking the tensor product of the individual states. So, for the provided expression, the tensor product is performed successively for each k as

$$|\psi_1\rangle \otimes |\psi_2\rangle \otimes \dots \otimes |\psi_n\rangle = (((|\psi_1\rangle \otimes |\psi_2\rangle) \otimes \dots) \otimes |\psi_n\rangle). \quad (15)$$

This composite state represents the joint quantum state of the entire system, where the Hilbert space $\mathcal{H}$ for the composite system is formed as the tensor product of the individual Hilbert spaces associated with each quantum system.

Qubits and Superpositions *Bits* are a common word for every computer scientist. On the machine level everything is based on the fundamental idea of bits. The following definitions for *qubits* are taken from Michael A. Nielsen and Isaac L. Chuang [6] unless stated otherwise.

A qubit can be described as a mathematical object with specific *states*. A qubit is in all states, but with different probabilities. A regular bit has either state 0 or 1. The two *canonical* states of the qubit are

$$\begin{aligned} |0\rangle &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ |1\rangle &= \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \end{aligned} \quad (16)$$

Here for $|0\rangle$ and $|1\rangle$, the first value in the vector can be interpreted as the "not-set" value and the second value stands for "set". Meaning that when these values then are representing the values of the qubits, when "not-set" is equal to 1, we have $|0\rangle$, and when "set" is 1, we have $|1\rangle$. These are the unit vectors from the first postulate of quantum computing. Important here is, that for qubits we will only be able to perform reversible operations.

This normalization ensures that the probabilities associated with different outcomes sum up to 1, providing a consistent and meaningful interpretation of quantum states.

Operations on $|0\rangle$, represented as a vector in $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$, using the unitary matrix can be summarized as below.

1. **Identity**, $f(b) = b$:

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad (17)$$

2. **Negation**, $f(b) = -b$:

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (18)$$

3. **Set to 0**, $f(b) = 0$:

$$\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad (19)$$

4. **Set to 1**, $f(b) = 1$:

$$\begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (20)$$

The same operations are also possible on $|1\rangle$, represented as a vector in $\begin{pmatrix} 0 \\ 1 \end{pmatrix}$ are also possible in the same way. Important here is to take a careful look at the matrices. The matrices used to perform the Identity and the Negation operation are both reversible, the ones used for setting the bit to either 0 or 1 are not. This also makes sense, as we can always negate a bit back to get to its initial value and the identity operation is just as in regular linear algebra comparable with a “NOP”, which is an operation that does not change any values. As stated, only operations with a reversible matrix here are supported for qubits. This is due to the *unitarity principle in quantum mechanics*. This principle in general demands an operation to be reversible, if it is applied again. Formally this is written via the unitary operation

$$U^T U = I. \quad (21)$$

This equation holds for the Identity (Equation (17)) and Negation operation (Equation (18)), but not for the arbitrary setting of the binary value in equation Equation (19) and equation Equation (20).

In the quantum world now, there no longer exists the simple identity of the bits. Instead there is

$$U|\psi\rangle = |\psi\rangle. \quad (22)$$

The operations remain the same for the canonical states. For U , we can choose the identity matrix such that operations satisfy the unitary principle. Choosing

$$U = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \text{and} \quad |\psi\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}, \quad (23)$$

will then yield

$$U|\psi\rangle = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}. \quad (24)$$

Qubits can also be in a *linear combination of states*, a so-called *superposition* of states (so infinitely many!). The superposition can be given via

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \alpha, \beta \in \mathbb{C}. \quad (25)$$

Superpositions also fulfill the four postulates of quantum mechanics which were described earlier. When measuring the state of the superposition, it "breaks". Then either the result is 0 with the probability $|\alpha|^2$, or 1 with probability $|\beta|^2$.

Example An example of a composite state $|0\rangle \otimes |0\rangle$. Written now with matrix notation as

$$|0\rangle \otimes |0\rangle = \begin{bmatrix} 1 & |0\rangle \\ 0 & |0\rangle \end{bmatrix} = \begin{bmatrix} 1 & \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ 0 & \begin{bmatrix} 1 \\ 0 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad (26)$$

1.3 Quantum Gates - CNOT and Hadamard Gates

Having now covered qubit states and superposition, it is now needed to check how operations inside a quantum system can be performed. For this subchapter we discuss two popular quantum gates, the the references are taken from Michael A. Nielsen and Isaac L. Chuang [6] unless stated otherwise.

CNOT Gates The *CNOT gate*, which stands for the *Controlled NOT gate*, is a two-qubit gate in quantum computing. It performs a conditional NOT operation on the target qubit (the second qubit) based on the state of the control qubit (the first qubit). If the control qubit is in the state $|1\rangle$, the target qubit's state is flipped; otherwise, it remains unchanged. When measuring the control qubit, it has no impact on the measurement outcome. In other words, whether the control qubit is in the state $|0\rangle$ or $|1\rangle$, measuring it will give the corresponding outcome with no influence from the CNOT gate itself. When measuring the target qubit, it reveals the result of the NOT operation if the control qubit is in state $|1\rangle$. The CNOT gate can be represented in a matrix notation as

$$U = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (27)$$

The four possible examples of applying the CNOT gate to a two-qubit state $|\psi_1\rangle|\psi_2\rangle$ are linked to Equation (26) and visualized in Equation (28).

$$U|00\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = |00\rangle \quad (28)$$

$$U|01\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = |01\rangle \quad (29)$$

$$U|10\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = |11\rangle \quad (30)$$

$$U|11\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = |10\rangle \quad (31)$$

Hadamard Gates The Hadamard gate is a quantum gate that puts a qubit into a superposition state. In simple terms, it transforms a qubit from a definite state (0 or 1) to a state where it exists in both 0 and 1 simultaneously. In the context of measurement, the Hadamard gate helps generate *superposition* states that can enhance the likelihood of obtaining the correct outcome for certain quantum algorithms. The Hadamard gate (H) transforms the basis state $|0\rangle$ into

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}} \quad (32)$$

(first column of H), representing a state 'halfway' between $|0\rangle$ and $|1\rangle$. This can be derived via

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 1 \end{pmatrix}_{\alpha} + \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \end{pmatrix}_{\beta}. \quad (33)$$

and also

$$\left(\frac{1}{\sqrt{2}}\right)^2 + \left(\frac{1}{\sqrt{2}}\right)^2 = 1. \quad (34)$$

This can be interpreted as

$$\left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}\right)^2 = \frac{1}{2} \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix} = I, \quad (35)$$

with

$$\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}. \quad (36)$$

Similarly, it transforms $|1\rangle$ into

$$\frac{|0\rangle - |1\rangle}{\sqrt{2}}, \quad (37)$$

which is also exactly in the middle between $|0\rangle$ and $|1\rangle$, which represents the concept of the superposition. In matrix notation, the Hadamard gate is represented as

$$H \equiv \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad (38)$$

It is important to note that H^2 is the identity gate, as simple algebra shows that $H^2 = I$, indicating that applying the Hadamard gate twice to a state results in no change. This also helps since for quantum computing it is required that all operations are reversible. Applying the Hadamard gate twice restores the original state.

If we take $H|\psi\rangle$ for

$$|\psi\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad (39)$$

we then get

$$\left(\frac{1}{\sqrt{2}}\right)^2 H \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \left(\frac{1}{\sqrt{2}}\right)^2 \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad (40)$$

which yields the final result

$$\frac{1}{2} \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}. \quad (41)$$

This then shows that

$$H^2|0\rangle = H(H|0\rangle) = |0\rangle. \quad (42)$$

The Hadamard gate is one of the most useful quantum gates in particular to instantiate the superposition in quantum circuits.

2 Shor's Algorithms

Shor proposed a Quantum algorithm for period finding. This algorithm can be used for both factorization and the DLP. The algorithm uses the *Quantum Fourier Transform* (QFT), which is the application of the *Discrete Fourier Transform* (DFT) in quantum computing, as seen in the paper from Fang Xi (Fangzhen) Lin [4]. The QFT can in particular cases speed up computations. Important applications of the QFT is the order finding of a periodic function, which we will see in Section 2.1, as well as how this technique can be applied to factor semiprime numbers. In Section 2.2 we will see how Shor's algorithm can be used to solve the DLP.

2.1 Factorization

The algorithm reduces the IFP to finding the *group order* of an element a modulo n in polynomial time. With some lemmata and proofs it can be shown, that the IFP can be solved by finding the order of a non-trivial element in $\mathbb{Z}/n\mathbb{Z}$.

Let $N = p \cdot q$. For some random $a \in \{0, \dots, N-1\}$, check if a is a factor of N . Define

$$f(x) := a^x \pmod{N}. \quad (43)$$

The idea is to now use the QFT to find the *period* of the periodic function f . That is the value ω such that

$$f(x + \omega) = f(x) \quad \forall x \in \mathbb{Z}. \quad (44)$$

In particular, this means that

$$f(0 + \omega) = f(0) = a^0 \equiv 1 \pmod{N}. \quad (45)$$

This furthermore implies that

$$f(\omega) = a^\omega \equiv 1 \pmod{N}. \quad (46)$$

That is, ω is the order, or a multiple of the order of a . Indeed, it is possible to retrieve the order r . For this we will need Lemma 1, taken from the paper published by Fang Xi Lin [4].¹

Lemma 1. *Let n be an odd integer, and let $n = p_1^{e_1} \cdot p_2^{e_2} \cdot \dots \cdot p_k^{e_k}$ be the prime factorization of n . Then the probability that a uniformly randomly chosen $x \in \mathbb{Z}/n\mathbb{Z}$ has even order r and $x^{r/2} \not\equiv -1 \pmod{n}$ is at least $1 - \frac{1}{2^{k-1}}$.*

In particular, if n is a semiprime, i.e. it is a product of two primes p, q , then the probability that r is even and $x^{r/2} \not\equiv -1 \pmod{n}$ is

$$1 - \frac{1}{2^{2-1}} = \frac{1}{2} = 0.5. \quad (47)$$

¹ For this lemma we do not state the proof as it has already been proven by Fang Xi Lin [4] and its lengthy proof does not contribute a lot to the overall derivation.

Since with a probability of 50% r is even and then $a^{\frac{r}{2}} \not\equiv -1 \pmod{N}$, we have

$$a^r \equiv 1 \pmod{N} \leftrightarrow a^r - 1 \equiv 0 \pmod{N}. \quad (48)$$

From this we also deduce that

$$(a^{\frac{r}{2}} + 1)(a^{\frac{r}{2}} - 1) \equiv 0 \pmod{N}. \quad (49)$$

Since we are interested in non-trivial factors, we require that $a^{\frac{r}{2}} + 1 \not\equiv 0 \pmod{N}$ and $a^{\frac{r}{2}} - 1 \not\equiv 0 \pmod{N}$. For the first factor, this means that

$$(a^{\frac{r}{2}} + 1) \not\equiv 0 \pmod{N} \leftrightarrow (a^{\frac{r}{2}}) \not\equiv -1 \pmod{N}, \quad (50)$$

which is already given by Lemma 1. For the second factor, we get

$$(a^{\frac{r}{2}} - 1) \not\equiv 0 \pmod{N} \leftrightarrow (a^{\frac{r}{2}}) \not\equiv 1 \pmod{N}, \quad (51)$$

which follows from the definition of the order. As a quick remark, the definition of the order tells us that

$$a^r \equiv 1 \pmod{N} \text{ and } \nexists 0 < k < r, \text{ such that } a^k \equiv 1 \pmod{N}. \quad (52)$$

Now the following Lemma by Fang Xi Lin [4] applies:

Lemma 2. *Given a composite number n , and x a non-trivial square root of 1 modulo n (i.e. $x^2 \equiv 1 \pmod{n}$) but x is neither 1 nor -1 modulo n , then either $\gcd(x-1, n)$ or $\gcd(x+1, n)$ is a non-trivial factor of n .*

Proof. Since $x^2 \equiv 1 \pmod{n}$, we have $x^2 - 1 \equiv 0 \pmod{n}$. Factoring, we get $(x-1)(x+1) \equiv 0 \pmod{n}$. This implies that n is a factor of $(x+1)(x-1)$. Since $x \pm 1 \not\equiv 0 \pmod{n}$, n has a non-trivial factor with $x+1$ or $x-1$. To find this common factor efficiently, we apply Euclid's (extended) algorithm to get $\gcd(x-1, n)$ or $\gcd(x+1, n)$, whereas $\gcd(x, y)$ represents the greatest common divisor of x and y . $\square$

Summarized, it gives us that

$$\underbrace{(a^{\frac{r}{2}})^2}_x = a^r \equiv 1 \pmod{N} \text{ and } a^{\frac{r}{2}} \not\equiv \pm 1 \quad (53)$$

All of this implies that n will be factored with probability $\frac{1}{2}$ in the first try!

The final algorithm can be taken, for example, from Ahlström and Dizdarevic [3, p. 16] and is restated in Algorithm 1.

Shor's algorithm exploits the period and order finding to find nontrivial factors efficiently. The $\gcd(a, b)$ is the greatest common divisor of a and b . For its computation there exists the (*Extended*) *Euclidian algorithm*. This algorithm is able to efficiently compute the gcd, also on a regular computer. The hard part of the above algorithm is its 5th step (Line 7 in Algorithm 1), as it needs the QFT and it relies on quantum computing.

Algorithm 1 Shor's Factoring Algorithm to factor (semiprime) $N = p \cdot q$

```

1: Input: Semiprime number  $N$ .
2: Output: Factors  $p$  and  $q$  of  $N$ .
3: Choose random  $a \in \{0, \dots, N-1\}$  and compute  $g = \gcd(a, N)$ 
4: if  $g \neq 1$  then
5:   We have found a factor of  $N$ , go to State 11.
6: end if
7: Find order  $r$  of  $a$ , i.e., find the period of  $f(x) = a^x \pmod{N}$ . ▷ Quantum Part
8: if  $r$  is odd or  $a^{r/2} \equiv -1 \pmod{N}$  then
9:   Bad luck  $\rightarrow$  start from the beginning
10: end if
11: Output  $\gcd(a^{r/2} - 1, N)$  or  $\gcd(a^{r/2} + 1, N)$  as a factor of  $N$ 

```

2.2 Recovering discrete logarithms for the Discrete Logarithm Problem (DLP)

Now it is time to analyze how Shor's algorithm can be utilized to solve the DLP.

Shor [8] distinguishes between an easy case for the DLP and a general case which is more involved. The easy case covers the scenario where $(p-1)$ is *smooth*. Smooth in this case means that the number only consists of small factors. This may seem intuitive and easy to check, but we are usually working with very large prime numbers for p . For example, having $p = 14567951311943370063315833900714950657$, we would now need to check for the factors of $p-1$. The factors of $p-1$ are then very small and it is easy to compute, in our example then $p-1 = 2^{87} \cdot 3^{23}$. The main reason for us just describing the easy case is that the general case is also based on the ideas of the easy one. For simplicity reasons therefore only the easy one will be discussed.

Let g be a generator of $\mathbb{Z}_p$ and let $x \in \mathbb{Z}_p$ such that $x \equiv g^r \pmod{p}$ for some unknown $r \in \mathbb{N}$. We want to recover r . For this, we define the DLP as a periodic function

$$f(a, b) := x^a g^{-b} \pmod{p}. \quad (54)$$

Again, the QFT is used to find the period of f . In this case the period needs two variables (ω_1, ω_2) such that

$$f(a + \omega_1, b + \omega_2) = f(a, b) \quad \forall a, b \in \mathbb{Z}. \quad (55)$$

The same principle as in Equation (45) holds, and in particular then

$$f(0 + \omega_1, 0 + \omega_2) = f(0, 0) \equiv x^0 g^0 \pmod{p} = 1 \pmod{p}. \quad (56)$$

This also implies that

$$f(\omega_1, \omega_2) = x^\omega g^{-\omega_2} \equiv 1 \pmod{p}. \quad (57)$$

Since $x \equiv g^r \pmod{p}$, we have

$$1 \equiv x^{\omega_1} g^{-\omega_2} \equiv (g^r)^{\omega_1} g^{-\omega_2} \pmod{p} \equiv g^{r\omega_1 - \omega_2} \pmod{p}. \quad (58)$$

From $g^{r\omega_1 - \omega_2} \equiv 1 \pmod{p}$ it follows by Fermat's little theorem that

$$r \cdot \omega_1 - \omega_2 \equiv 0 \pmod{\phi(p)}, \quad (59)$$

whereas $\phi(p) = p - 1$, since p is prime. In particular,

$$r \cdot \omega_1 \equiv \omega_2 \pmod{p - 1}. \quad (60)$$

If ω_1 is invertible mod $p - 1$, that is, $\gcd(\omega_1, p - 1) = 1$ and $\omega_2 \neq 0$, then we can calculate r directly:

$$r = \omega_2 \cdot \omega_1^{-1} \pmod{p - 1}. \quad (61)$$

Combining all observations gives the algorithm as demonstrated in Algorithm 2.

Algorithm 2 Discrete Logarithm Problem (DLP)

- 1: **Input:** Prime p , generator g , $x \equiv g^r \pmod{p}$
 - 2: **Output:** We want to compute r given x, g, p
 - 3: We define a periodic function $f(a, b) = x^a g^{-b}$
 - 4: Find the order (period), a, b , s.t. $(x^a g^{-b} \equiv 1) \pmod{p}$ ▷ Quantum Part
 - 5: If $f(a + w_1, b + w_2) = f(a, b)$, then (w_1, w_2) is the period (order)
 - 6: Then it holds that $x^{a+w_1} g^{-b-w_2} = x^a x^{w_1} g^{-b} g^{w_2}$, where $x^a g^{-b}$ then cancels out and $x^{w_1} g^{w_2} \equiv 1 \pmod{p}$
 - 7: Having now a temporary result $v = a^k$, where a is a function parameter and k might be a multiple of the order, we can derive $a^{kw_1 - w_2} \equiv 1 \pmod{p}$, where $kw_1 - w_2$ is the wanted order r
 - 8: This then gives $kw_1 - w_2 \equiv 0 \pmod{\phi(p)}$
-

2.3 Recovering discrete logarithms with the Number Field Sieve (NFS) approach

This time we want to dive into the algorithm for the *(General) Number Field Sieve*, which does not rely on quantum computing, to see another approach for recovering discrete logarithms, using nothing but number theory and a regular computer. For this chapter we rely on the paper from Michael Case [2] unless stated otherwise.

Suppose that one wants to factor a composite number n , and for two numbers $s, r \in \mathbb{Z}$, $s^2 \equiv r^2 \pmod{n}$. Then $s^2 - r^2 \equiv 0 \pmod{n}$. Suppose n has the prime factorization $n = pq$. Then

$$pq \mid (s^2 - r^2) \quad (62)$$

This implies that

$$p \mid (s-r)(s+r) \quad \text{and} \quad q \mid (s-r)(s+r), \quad (63)$$

which represents a fundamental principle from number theory.

If c divides ab and the greatest common divisor of b and c is 1, then c must divide a .

This leads to the following conditions:

$$\begin{cases} p \mid (s+r) \text{ or } p \mid (s-r) \\ q \mid (s+r) \text{ or } q \mid (s-r), \end{cases} \quad (64)$$

which holds because p, q are primes (i.e. not composite). These conditions imply that it is impossible for both p and $(s+r)$ or p and $(s-r)$ to be coprime. Similarly, it is impossible for both q and $(s+r)$ or q and $(s-r)$ to be coprime. We can summarize our findings in Table 1:

Possible Divisibility Scenarios				GCD Results		Successful Factorization
$p \mid (s+r)$	$p \mid (s-r)$	$q \mid (s+r)$	$q \mid (s-r)$	$\gcd(pq, s+r)$	$\gcd(pq, s-r)$	
No	Yes	No	Yes	0	pq	
No	Yes	Yes	No	q	p	*
No	Yes	Yes	Yes	q	pq	*
Yes	No	No	Yes	p	q	*
Yes	No	Yes	No	pq	0	
Yes	No	Yes	Yes	pq	q	*
Yes	Yes	No	Yes	p	pq	*
Yes	Yes	Yes	No	pq	p	*
Yes	Yes	Yes	Yes	pq	pq	

Table 1: Possibilities for p and q dividing $s+r$ and $s-r$, taken from (page 3) [2]

The table summarizes the potential scenarios for p and q dividing $s+r$ and $s-r$. As an example, considering $p \mid (s+r)$, $p \mid (s-r)$, $q \nmid (s+r)$, and $q \mid (s-r)$, the possible values for $\gcd(pq, s+r)$ lie within $\{1, p, q, pq\}$, the divisors of $n = pq$. Since $p \mid (s+r)$, $p \mid \gcd(pq, s+r)$. Subsequently, $pq - (s+r)$ because $q - (s+r)$, leading to $\gcd(pq, s+r)$ assuming the value p . Similarly, $\gcd(pq, s-r) = pq$ due to both $p \mid (s-r)$ and $q \mid (s-r)$. This situation successfully isolates either p or q , leading to the factorization of $n = pq$.

Assuming equal likelihood of all combinations in the table, if $s^2 \equiv r^2 \pmod{n}$, it implies that either $\gcd(pq, s+r)$ or $\gcd(pq, s-r)$ yields a nontrivial factor of $n = pq$ with a probability of $2/3$. Although having $s^2 \equiv r^2 \pmod{n}$ does not guarantee a nontrivial factorization of n , the high probability suggests that finding pairs s, r satisfying $s^2 \equiv r^2 \pmod{n}$ should not require many attempts to factor n .

2.4 Comparison to Shor's Algorithm and Analysis

We can here take the words from Michael Case's conclusion of his paper about the Number Field Sieve (NFS) and General Number Field Sieve (GNFS) [2]:

The GNFS is a very sophisticated algorithm for factoring composite numbers. While the algorithm is complex, it does successfully factor a number relatively quickly. Due to the setup steps necessary, the GNFS is significantly slower than other popular factoring methods for small composite numbers. However, for large composite numbers the time spent in setting up the GNFS is negligible and the algorithm is dramatically faster than any other factoring algorithm. This point was proved on January 18, 2002 when a team of researchers from the University of Bonn successfully factored a 155 digit (512 bit) composite integer in 3.7 months using the GNFS. Had this 512 bit number been a public key in the RSA cryptosystem, the security of the system would have been compromised and an adversary would have gained access to the private key. The development of the GNFS has brought into question the security of the RSA cryptosystem. Because of the widespread use of RSA, the existence of the GNFS should cause any computer security expert to worry.

We will now very briefly compare this to Shor's original algorithm. By the help of Vladimir Silva [9] we can analyze both algorithms quickly together and even state some $\mathcal{O}$ -notations for the time complexity. Important here is to mention that the runtime used for Shor's algorithm is purely theoretical, as quantum computers are still widely inaccessible and too unstable to perform Shor's algorithm in a reliable way.

According to Silva's research, Shor's algorithm provides a significant speedup over the NFS in factoring large numbers that form the foundation of current cryptography. Its practical implementation could render current asymmetric encryption methods useless.

Shor's algorithm exhibits exponential superiority compared to classical algorithms (such as NFS) for factoring large numbers, particularly those used in cryptographic keys. This exponential advantage poses a significant threat to the security of current cryptographic systems.

The most important aspects of his publication the runtime and memory requirements of the discussed algorithms to find the factorization of large (semiprime) numbers, which are also used in RSA. However the image uploaded on Silva's publication [9] seems to have a typo regarding the expected runtime of Shor's algorithm for recovering discrete logarithms. As seen in Lin's paper [4](page 15), the runtime for Shor's algorithm is $\mathcal{O}(\log^3 n)$, therefore the runtime for Shor's algorithm was "adapted" for the graph. Also, the graph below is purely theoretical and is missing concrete measurement units, as stated by Lin [4], the IBM group was only really able in 2001 to factor the small semiprimes 15 and 21, even there with doubt of the actual correct process of Shor's algorithm.

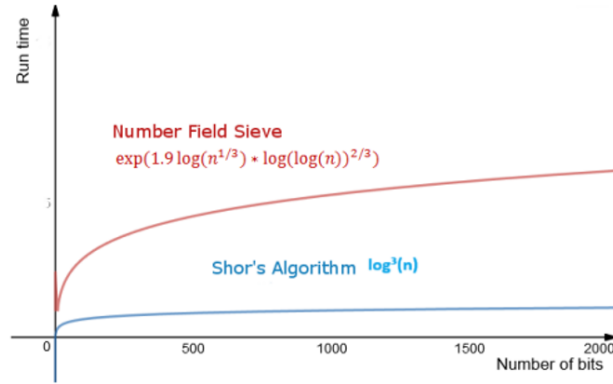


Fig. 1: Comparison between NFS and Shor's algorithm

3 Discussion

Shor's original paper is already known to the public for 20 years by now. The impact it left and still has to this day is quite enormous and respectful. While many claim that quantum computing is still far from achievable, the threat is still there. As seen in the case of the NFS there are already steps into non-quantum algorithms which might be able to break the DLP and IFP without even needing the help of qubits. With us now only having covered a very small fraction of the mathematics and the ongoing research in the field of quantum computing and quantum cryptography, we cannot make a decisive final statement which is guaranteed to hold. Somewhere in the near future the threat might become a real problem to everyone relying on the DH key exchange and the RSA encryption and decryption protocol. The best we can do is to hope that until that day there will be already secure alternatives and that research in this field will still be ongoing in the future.

References

1. Boneh, D., Joux, A., Nguyen, P.Q.: Why textbook ElGamal and RSA encryption are insecure. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 30–43. Springer (2000)
2. Case, M.: A beginner's guide to the general number field sieve. pp. 2–18
3. Dizdarevic, P., Ahlström, M.: Mathematical analysis and simulation of shor's algorithm and the quantum fourier transform (2012)
4. Lin, F.X.: Shor's algorithm and the quantum fourier transform. pp. 2, 4–6, 9, 12, 14–15 (2014)
5. Merkle, R.C.: Secure communications over insecure channels. *Commun. ACM* **21**, 294–299 (1978), <https://api.semanticscholar.org/CorpusID:6967714>, accessed: 2024-04-28
6. Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information (10th Anniversary edition) (2011), <https://api.semanticscholar.org/CorpusID:263408726>
7. Rivest, R.L., Shamir, A., Adleman, L.M.: A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM* **26**, 96–99 (1978), <https://api.semanticscholar.org/CorpusID:30798417>
8. Shor, P.W.: Algorithms for quantum computation: discrete logarithms and factoring. *Proceedings 35th Annual Symposium on Foundations of Computer Science* pp. 124–134 (1994), <https://api.semanticscholar.org/CorpusID:15291489>, accessed: 2024-04-28
9. Silva, V.: Large integer factorization: Number field sieve vs shor (Aug 2019), <https://sharkyspool.home.blog/2019/08/02/large-integer-factorization-number-field-sieve-vs-shor/>, accessed: 2024-04-28